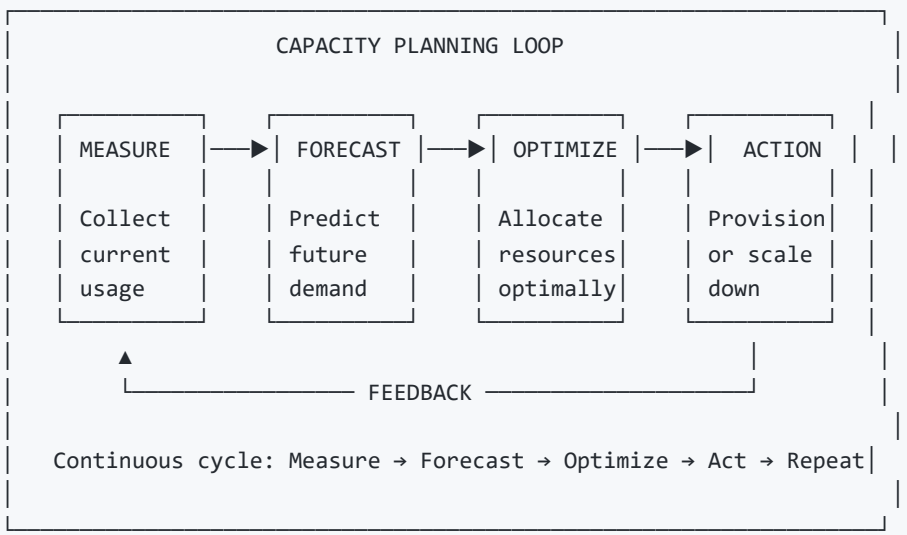
Capacity Planning, Data Platforms & AI-Assisted Engineering — Interview Guide

> 📋 **Purpose**: Deep-dive reference for interview discussions on capacity planning, data platforms, ML/AI systems, and AI-assisted engineering patterns.

Part 1 — Capacity Planning, Forecasting & Optimization

What Is Capacity Planning?

Capacity planning is the process of determining the resources (compute, storage, network, people) needed to meet future demand — ensuring you have **enough** capacity to serve workloads without **over-provisioning** (wasting money).



The Four Pillars of Capacity Planning

Pillar	What You Plan	Key Metrics	Example
Compute	CPU, GPU, containers, VMs	CPU utilization, request latency, queue depth	"We need 200 more VMs by Q3"
Storage	Disk, blob, database	Storage used vs provisioned, IOPS,	"Database will hit 80% in 6 weeks"

		throughput	
Network	Bandwidth, connections	Throughput, packet loss, connection count	"CDN egress growing 15% monthly"
Cost	Cloud spend, hardware budget	$/request, $/GB, reserved vs on-demand mix	"Shift 60% to reserved instances to save 40%"

Key Concepts

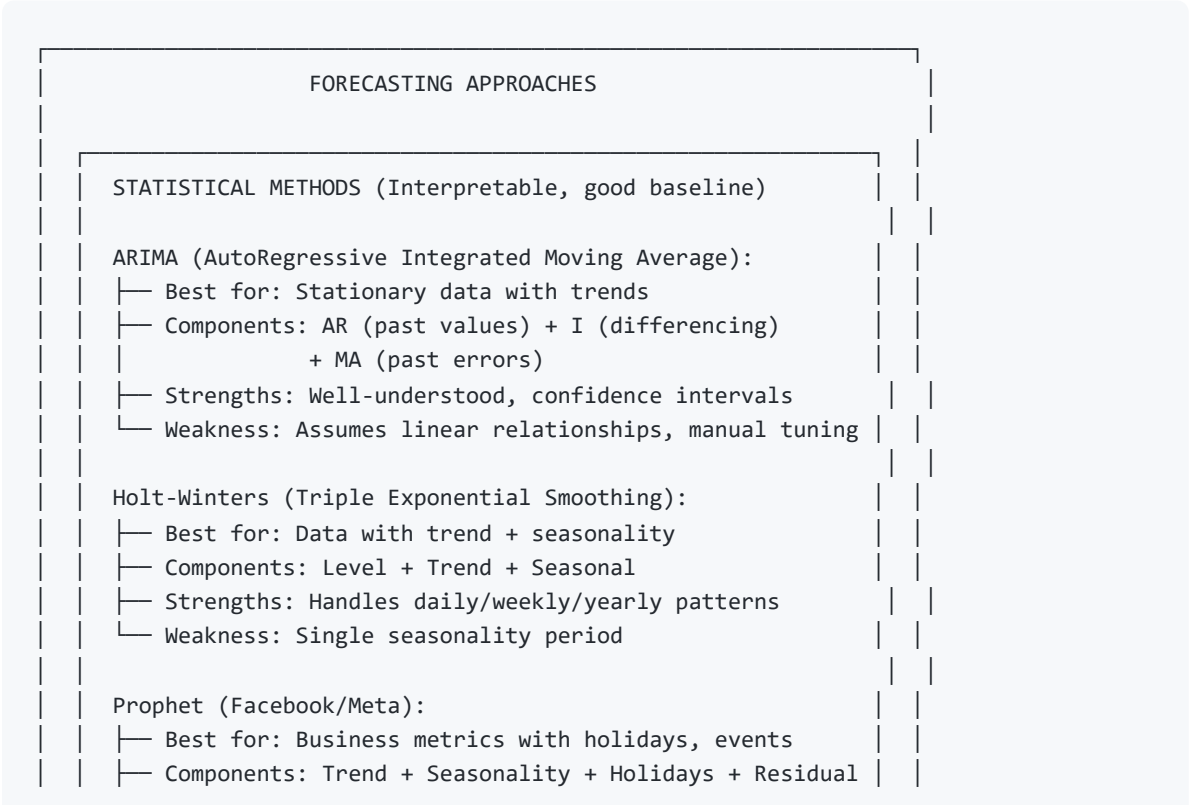
Headroom — Buffer between current usage and max capacity. Typical target: 30-40% headroom for production systems. Too little = risk of outage. Too much = wasted spend.

Lead Time — Time between deciding to add capacity and it being available. Cloud: minutes (auto-scale). Physical hardware: weeks to months (procurement). Long lead times require earlier and more accurate forecasts.

Saturation Point — The load level where performance degrades non-linearly (latency spikes, errors increase). Systems often collapse well before 100% utilization — plan for 70-80% max.

Right-Sizing — Matching resource allocation to actual usage. Common finding: 40-60% of cloud VMs are over-provisioned. Right-sizing saves significant cost without impacting performance.

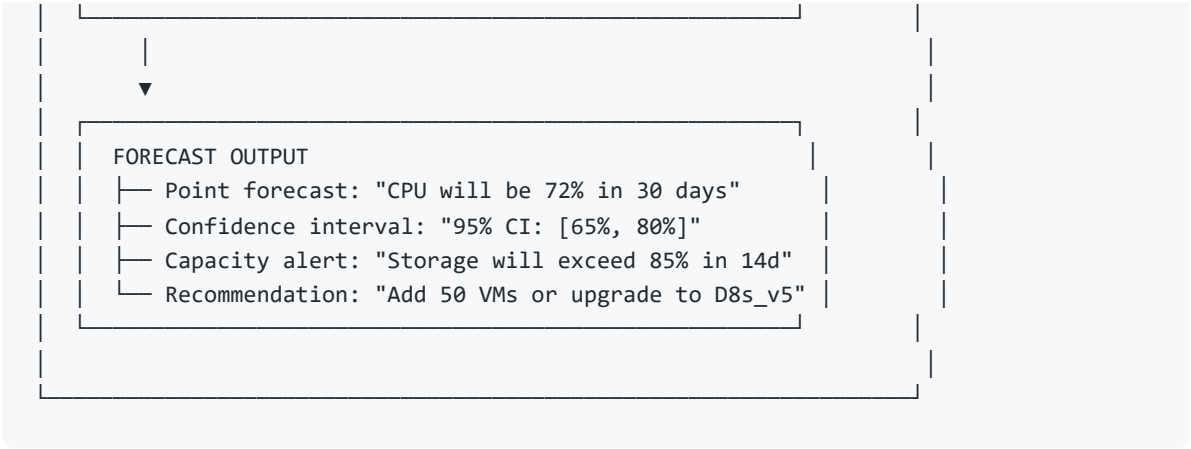
Forecasting Methods



```
│  ├── Strengths: Auto-handles missing data, changepoints    │  │
│  └── Weakness: Not great for high-frequency (sub-hourly)   │  │
└──────────────────────────────────────────────────────────────┘  │
                                                                  │
┌──────────────────────────────────────────────────────────────┐  │
│  ML-BASED METHODS (More powerful, less interpretable)        │  │
│                                                              │  │
│  Gradient Boosted Trees (XGBoost, LightGBM):                 │  │
│  ├── Treat forecasting as regression with lag features       │  │
│  ├── Can incorporate external features (holidays, events)    │  │
│  └── Good for: Tabular data with many correlated signals     │  │
│                                                              │  │
│  Neural Networks (LSTM, Temporal Fusion Transformers):       │  │
│  ├── Learn complex non-linear temporal patterns              │  │
│  ├── Good for: Large datasets, multiple related series       │  │
│  └── Weakness: Need more data, harder to interpret           │  │
│                                                              │  │
│  Ensemble Methods:                                           │  │
│  ├── Combine multiple models (statistical + ML)              │  │
│  ├── Weighted average based on recent accuracy               │  │
│  └── Often the best approach: no single model wins always    │  │
└──────────────────────────────────────────────────────────────┘  │
                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

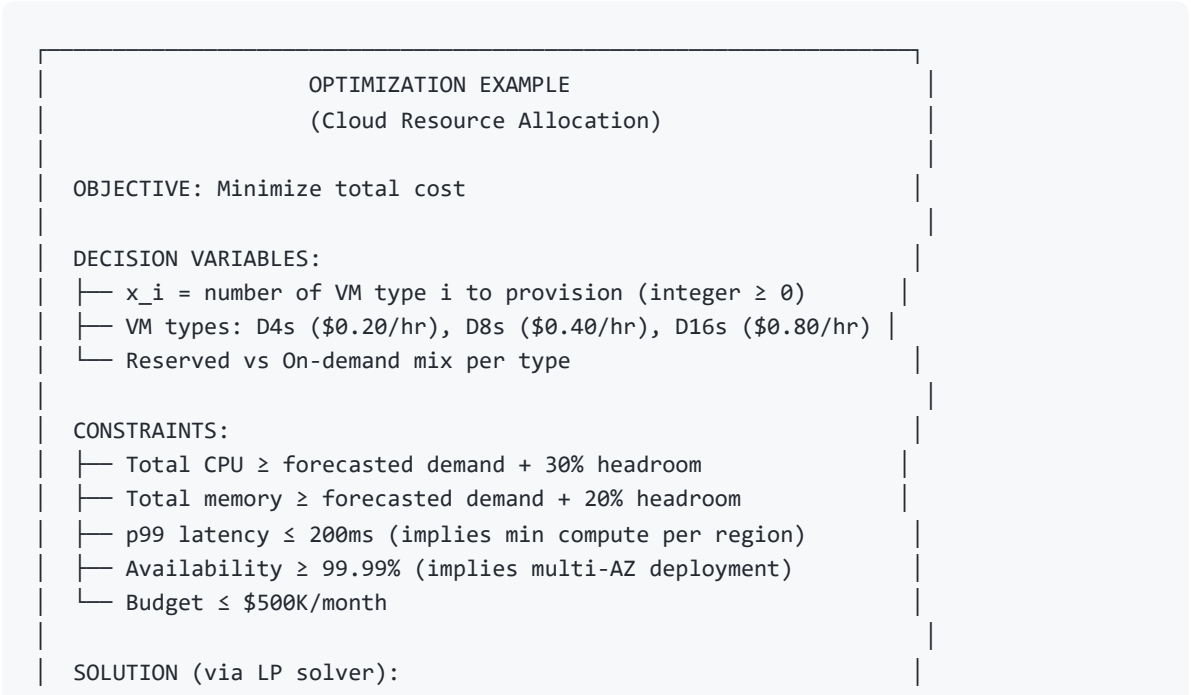
Forecasting Pipeline

```
┌──────────────────────────────────────────────────────────────────┐
│                    FORECASTING PIPELINE                          │
│                                                                  │
│  RAW METRICS                                                     │
│  ├── CPU utilization (per host, per cluster)                     │
│  ├── Request count / QPS                                         │
│  ├── Storage usage (bytes, IOPS)                                 │
│  ├── Network throughput                                          │
│  └── Business signals (user signups, orders, campaigns)          │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────┐            │
│  │  DATA PREPARATION                                │            │
│  │  1. Aggregate to consistent intervals (5min, 1hr, 1d)        │
│  │  2. Handle missing values (interpolation, forward-fill)      │
│  │  3. Remove outliers / anomalies (Z-score, IQR filter)        │
│  │  4. Add calendar features (day-of-week, holidays)            │
│  │  5. Add lag features (value 1h ago, 1d ago, 1w ago)          │
│  └──────────────────────────────────────────────────┘            │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────┐            │
│  │  MODEL TRAINING                                  │            │
│  │  ├── Train multiple models (ARIMA, Prophet, XGBoost)         │
│  │  ├── Walk-forward validation (train on past, test next)      │
│  │  ├── Metric: MAPE, RMSE, coverage of prediction interval     │
│  │  └── Select best model per metric/resource type              │
```



Optimization Techniques

Technique	What It Solves	Example
Linear Programming (LP)	Minimize cost subject to constraints	Minimize cloud spend while meeting SLA latency targets
Mixed Integer Programming (MIP)	LP with discrete decisions (on/off)	Choose which VM SKUs to use (can't use 0.7 of a VM)
Bin Packing	Fit items into minimum containers	Pack workloads onto fewest servers (VM placement)
Constraint Satisfaction	Find any valid configuration	Find a deployment that meets all SLA + budget + region constraints
Multi-Objective Optimization	Balance competing goals	Minimize cost AND latency AND carbon footprint simultaneously



```
│  ├── 120× D8s reserved (best $/CPU for steady load)            │
│  ├── 30× D16s on-demand (for peak bursts)                      │
│  ├── Total cost: $420K/month (16% under budget)                │
│  └── Headroom: 35% CPU, 28% memory                             │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

Decision Support Systems

A decision support system (DSS) helps humans make better decisions by presenting data, forecasts, and recommendations in actionable form.

```
┌────────────────────────────────────────────────────────────────┐
│                  DECISION SUPPORT SYSTEM LAYERS                │
│                                                                │
│  ┌──────────────────────────────────────────────────────┐      │
│  │  LAYER 4: ACTIONS                                    │      │
│  │  Automated: Auto-scale, auto-provision, auto-rebalance      │
│  │  Human: Approve purchase order, change architecture  │      │
│  └──────────────────────────────────────────────────────┘      │
│                          ▲                                     │
│  ┌──────────────────────────────────────────────────────┐      │
│  │  LAYER 3: RECOMMENDATIONS                            │      │
│  │  "Add 50 D8s VMs in East US by March 15"             │      │
│  │  "Migrate cold storage to archive tier to save $30K/mo"     │
│  │  Confidence score + cost impact + risk assessment    │      │
│  └──────────────────────────────────────────────────────┘      │
│                          ▲                                     │
│  ┌──────────────────────────────────────────────────────┐      │
│  │  LAYER 2: ANALYTICS & FORECASTS                      │      │
│  │  "CPU will reach 85% in 21 days (95% CI: 15-28 days)"       │
│  │  What-if simulator: "If traffic grows 2x, we need..."       │
│  │  Trend analysis, anomaly detection, correlation      │      │
│  └──────────────────────────────────────────────────────┘      │
│                          ▲                                     │
│  ┌──────────────────────────────────────────────────────┐      │
│  │  LAYER 1: DATA & DASHBOARDS                          │      │
│  │  Real-time metrics, historical trends, utilization maps     │
│  │  Alerting on thresholds (CPU > 80%, storage > 85%)   │      │
│  └──────────────────────────────────────────────────────┘      │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

Interview Talking Points — Capacity Planning

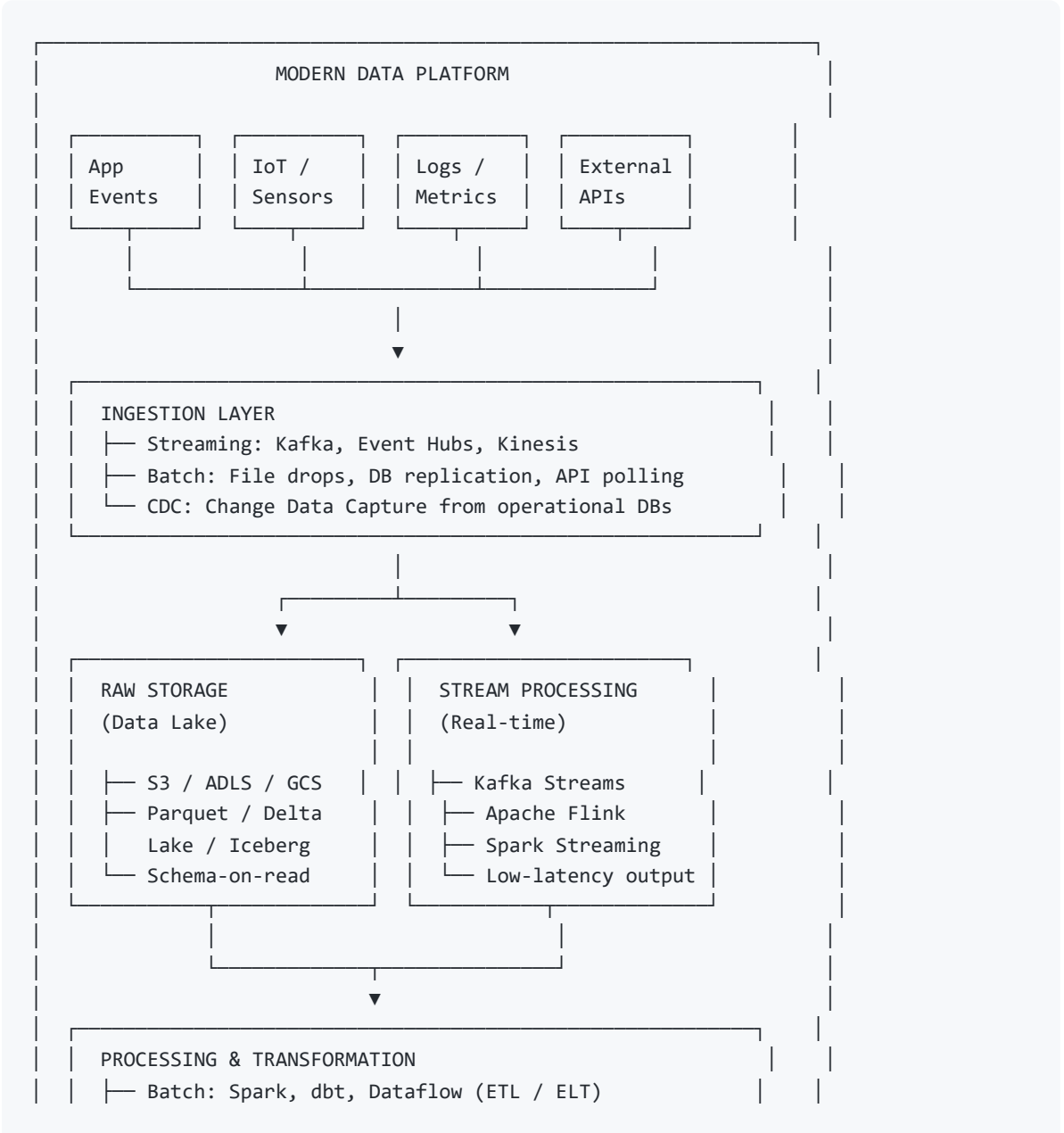
- "Capacity planning is a **continuous feedback loop**: measure current usage, forecast future demand using time-series models, optimize resource allocation, act on the recommendations, then measure again."
- "I'd use **Prophet or ARIMA** for baseline forecasting because they handle seasonality well, then layer in **ML models** (XGBoost) when we have enough correlated features like business

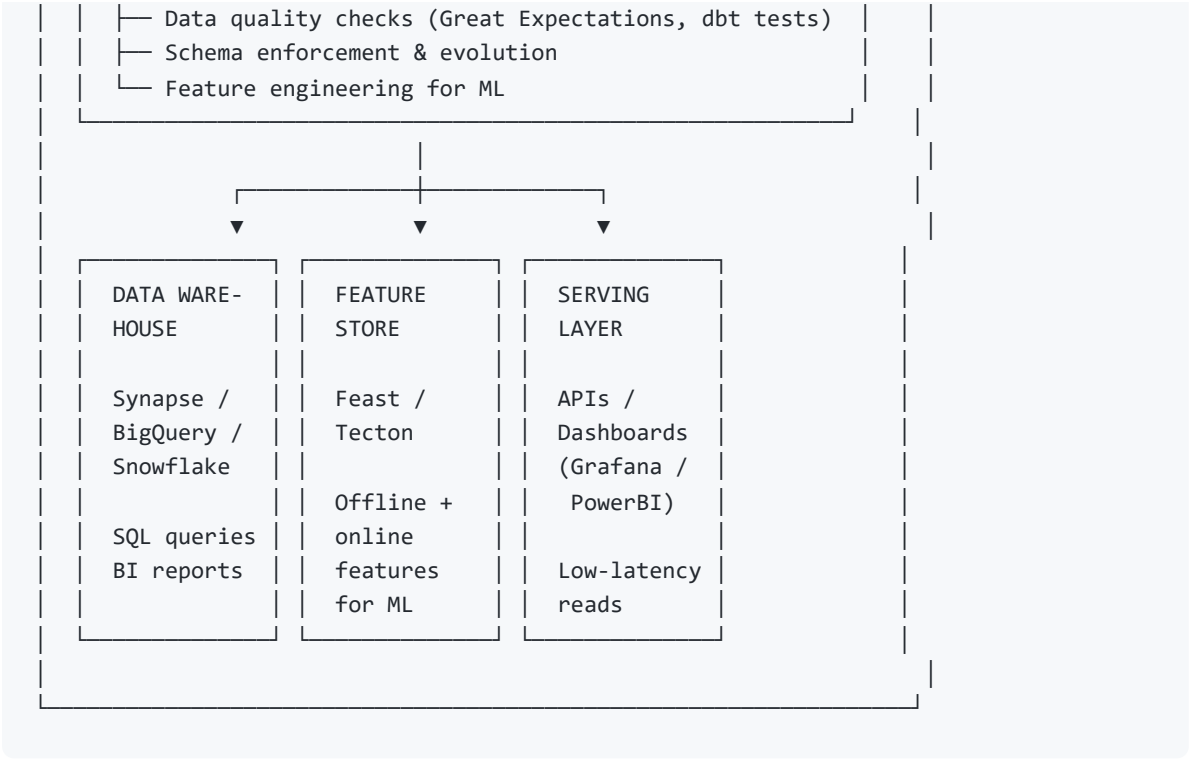
events and marketing campaigns."

- "The key trade-off is **headroom vs cost**: too much headroom wastes money, too little risks outages. I target 30-40% headroom for critical systems and use confidence intervals to quantify the risk."
- "Optimization isn't just 'buy more servers' — it's a **constraint satisfaction problem**: minimize cost while meeting SLA, latency, availability, and regional requirements. LP/MIP solvers formalize this."

Part 2 — Data Platforms & Analytics Pipelines

Data Platform Architecture



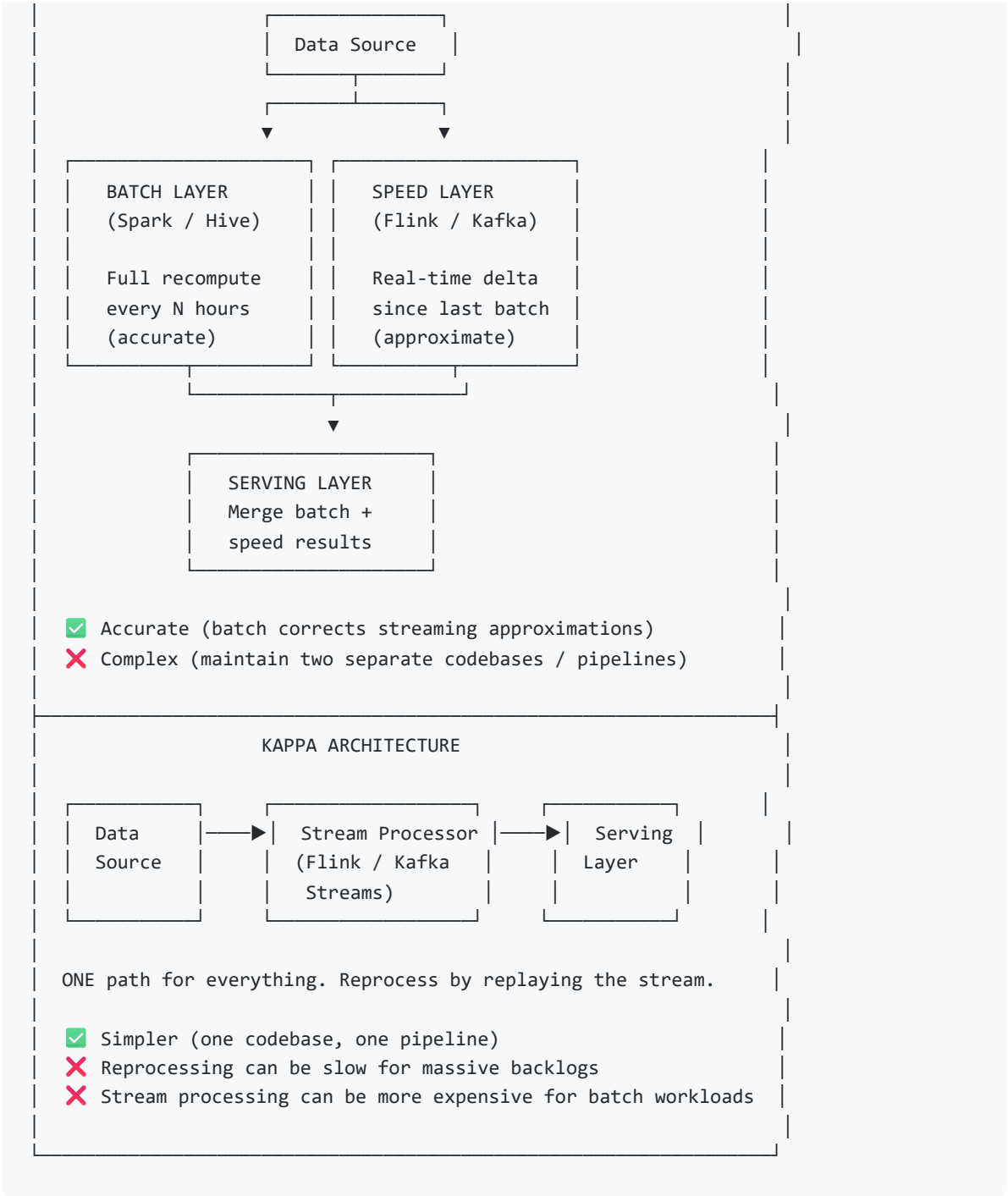


Batch vs Streaming

Aspect	Batch Processing	Stream Processing
Latency	Minutes to hours	Milliseconds to seconds
Data	Bounded dataset (files, tables)	Unbounded, continuous events
Tools	Spark, Hive, dbt, Dataflow	Kafka Streams, Flink, Spark Structured Streaming
Use Case	Daily reports, model training, backfills	Real-time dashboards, alerting, live features
Fault Tolerance	Retry entire job	Checkpointing + exactly-once semantics
Cost	Pay per job (cheaper for large volumes)	Always-on (pay for running infra)
Complexity	Simpler (process complete data)	Harder (ordering, late data, windowing)

Lambda vs Kappa Architecture





Data Lake vs Data Warehouse vs Lakehouse

Aspect	Data Lake	Data Warehouse	Lakehouse
Storage	Object store (S3, ADLS)	Proprietary (Snowflake, BigQuery)	Object store + table format
Format	Raw (JSON, CSV, Parquet)	Structured tables	Open table formats (Delta, Iceberg)

Schema	Schema-on-read	Schema-on-write	Schema-on-write with evolution
Query	Spark, Presto	SQL	SQL + Spark
ACID	No	Yes	Yes (via Delta Lake / Iceberg)
Cost	Cheapest storage	More expensive	Cheap storage + warehouse features
Best For	Raw data, ML training	BI, reporting, dashboards	Both — unified platform
Example	S3 + Athena	Snowflake	Databricks (Delta Lake)

ETL vs ELT

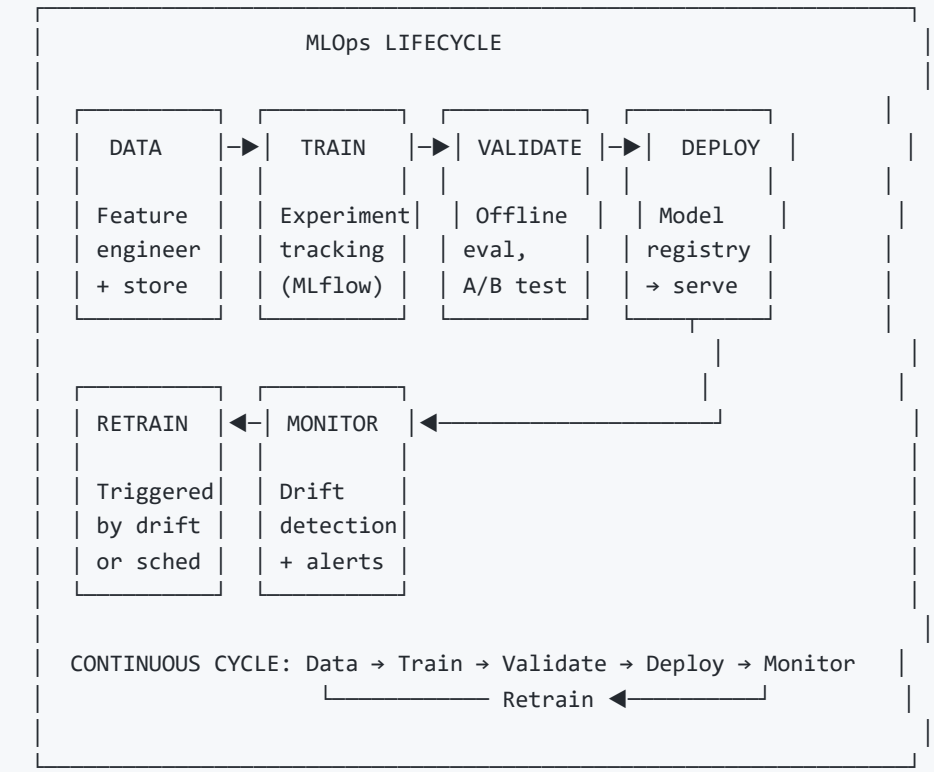
```
┌──────────────────────────────────────────────────────────────┐
│  ETL (Extract → Transform → Load)                            │
│                                                              │
│  Source → [Transform in pipeline] → Load clean data to warehouse │
│  ├── Transform happens BEFORE loading                        │
│  ├── Smaller data in warehouse (only clean data)             │
│  ├── Traditional approach (Informatica, Talend)              │
│  └── Harder to change transformations retroactively          │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│  ELT (Extract → Load → Transform)                            │
│                                                              │
│  Source → Load raw data to lake/warehouse → [Transform in-place] │
│  ├── Transform happens AFTER loading (using warehouse compute)   │
│  ├── Raw data preserved (can re-transform)                   │
│  ├── Modern approach (dbt + Snowflake/BigQuery/Databricks)   │
│  └── Leverage warehouse's powerful compute for transformations   │
│                                                              │
│  INDUSTRY TREND: ELT is winning. dbt is the standard tool.   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Data Quality, Lineage & Governance

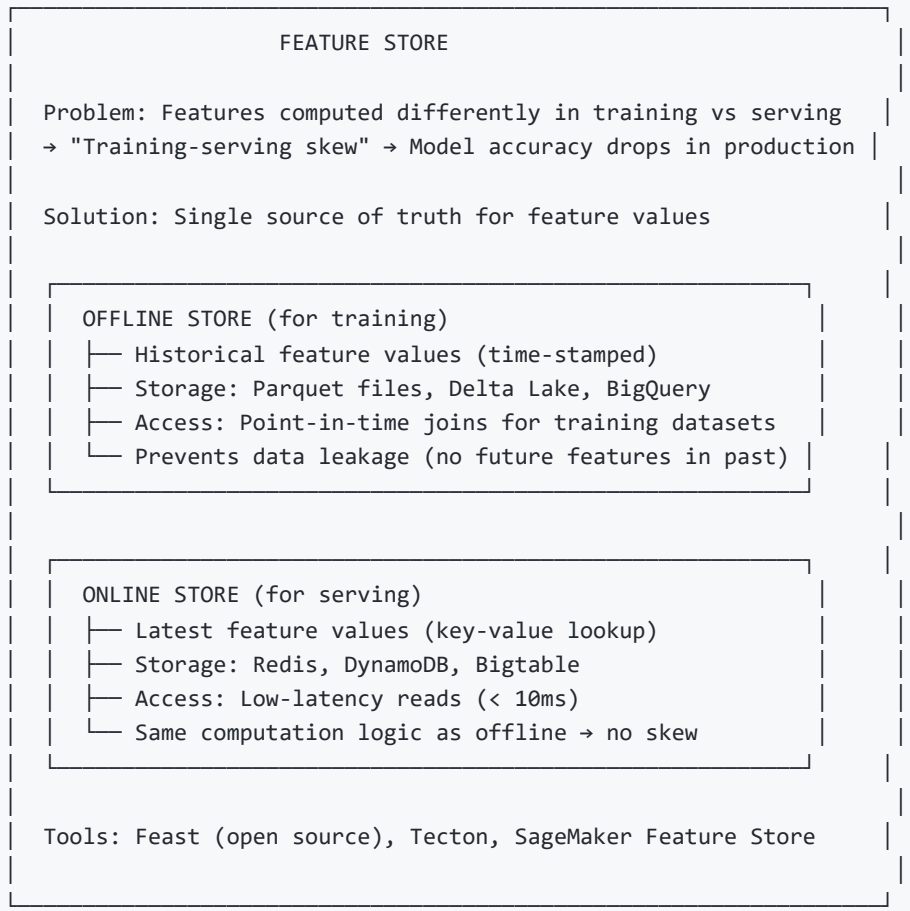
```
┌──────────────────────────────────────────────────────────────┐
│                    DATA QUALITY                              │
│                                                              │
│  DIMENSIONS:                                                 │
│  ├── Completeness — Are all expected records present?        │
│  ├── Accuracy — Do values match reality?                     │
│  ├── Freshness — Is data up-to-date?                         │
```

```
│  ├── Consistency — Same data across systems?                    │
│  ├── Uniqueness — No unwanted duplicates?                       │
│  └── Schema conformance — Correct types, formats?               │
│                                                                  │
│  TOOLS:                                                         │
│  ├── Great Expectations — Define and test data "expectations"   │
│  ├── dbt tests — SQL-based assertions (unique, not_null, etc.)  │
│  ├── Monte Carlo / Soda — Automated anomaly detection on data   │
│  └── Custom checks — Business-rule validation in pipeline       │
│                                                                  │
│  DATA LINEAGE:                                                  │
│  ├── "Where did this data come from?"                           │
│  ├── Track: Source → Transformations → Downstream consumers     │
│  ├── Tools: Apache Atlas, dbt docs, OpenLineage, Datahub        │
│  └── Critical for debugging, compliance (GDPR), impact analysis │
│                                                                  │
│  DATA GOVERNANCE:                                               │
│  ├── Access control (who can read/write what data)              │
│  ├── Classification (PII, confidential, public)                 │
│  ├── Retention policies (how long to keep data)                 │
│  └── Audit trails (who accessed what, when)                     │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

ML Model Integration & Lifecycle (MLOps)



Feature Store



Model Serving Patterns

Pattern	Description	Latency	Use Case
Online (real-time)	Model behind REST/gRPC API, scored per request	< 100ms	Fraud detection, recommendation, ranking
Batch	Score entire dataset on schedule (daily/hourly)	Minutes	Email campaigns, risk scoring, capacity forecasts
Streaming	Model embedded in stream processor, scores each event	< 1s	Anomaly detection, real-time alerting
Edge	Model runs on device (phone, IoT, browser)	< 10ms	Voice assistant, on-device image recognition
Shadow	New model runs alongside old; traffic to both, only old serves	N/A	Safe rollout, A/B comparison without risk

Model Drift Detection

```
┌──────────────────────────────────────────────────────────────┐
│                    MODEL DRIFT                               │
│                                                              │
│  DATA DRIFT: Input distribution changes                      │
│  ├── Example: Training data had 20% mobile traffic,          │
│  │   production now has 60% mobile                           │
│  ├── Detection: KS test, PSI (Population Stability Index),   │
│  │   distribution comparison on each feature                 │
│  └── Fix: Retrain on recent data                             │
│                                                              │
│  CONCEPT DRIFT: Relationship between input and output changes│
│  ├── Example: COVID changed purchasing patterns →            │
│  │   demand forecast model predictions became wrong          │
│  ├── Detection: Monitor prediction accuracy over time        │
│  │   (MAPE increasing, accuracy decreasing)                  │
│  └── Fix: Retrain with new data, possibly new features       │
│                                                              │
│  MONITORING APPROACH:                                        │
│  ├── Log all predictions + ground truth (when available)     │
│  ├── Compute metrics on sliding windows (hourly, daily)      │
│  ├── Alert when metrics degrade beyond threshold             │
│  ├── Auto-trigger retraining pipeline if drift confirmed     │
│  └── Tools: Evidently AI, WhyLabs, custom Prometheus metrics │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

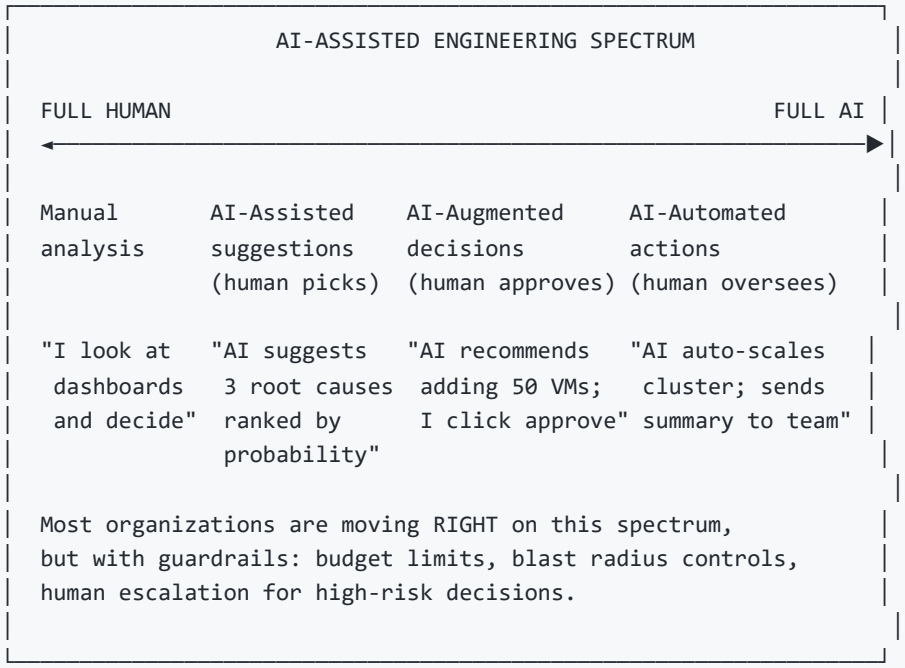
Interview Talking Points — Data Platforms

- "A modern data platform has four layers: **ingestion** (Kafka/CDC), **storage** (data lake in Parquet/Delta), **processing** (Spark/dbt for ELT), and **serving** (warehouse for BI, feature store for ML, APIs for applications)."
- "I prefer **ELT over ETL** because raw data is preserved in the lake — you can re-transform when requirements change without re-ingesting."
- "**Feature stores** solve training-serving skew: same feature computation logic for offline training and online serving. Without one, you get subtle bugs where model accuracy in production is worse than in experiments."
- "MLOps is about treating ML models like software: **version everything** (data, code, model, config), **test before deploy** (offline evaluation + shadow deployment), **monitor in production** (data drift, concept drift, accuracy decay), and **automate retraining** when drift is detected."

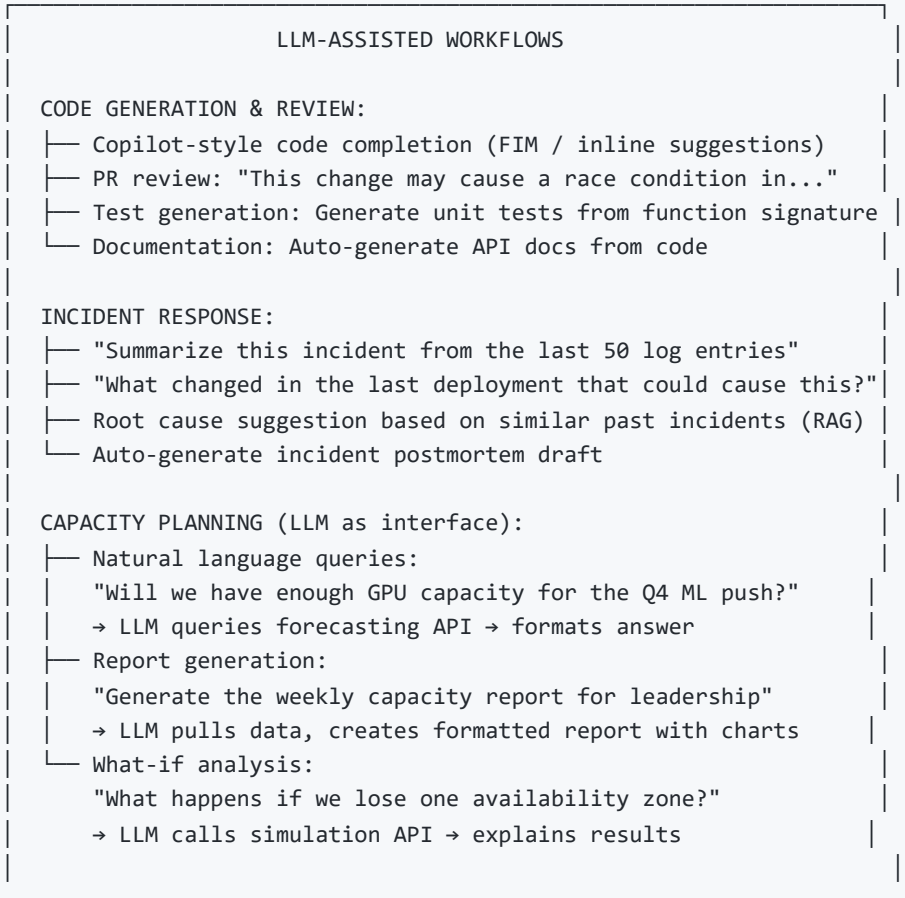
Part 3 — AI-Assisted Engineering Patterns

Overview

AI-assisted engineering uses LLMs and predictive models to **augment** (not replace) human decision-making in engineering workflows. The key pattern is: **AI suggests, human decides**.



Pattern 1: LLMs for Engineering Workflows



```
│  KEY PATTERN: LLM as an orchestrator (calls tools/APIs)          │
│  ├── LLM understands intent from natural language               │
│  ├── Calls structured APIs for data (no hallucination risk)     │
│  ├── Formats results for human consumption                      │
│  └── This is the "Agent" pattern (ReAct loop)                   │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

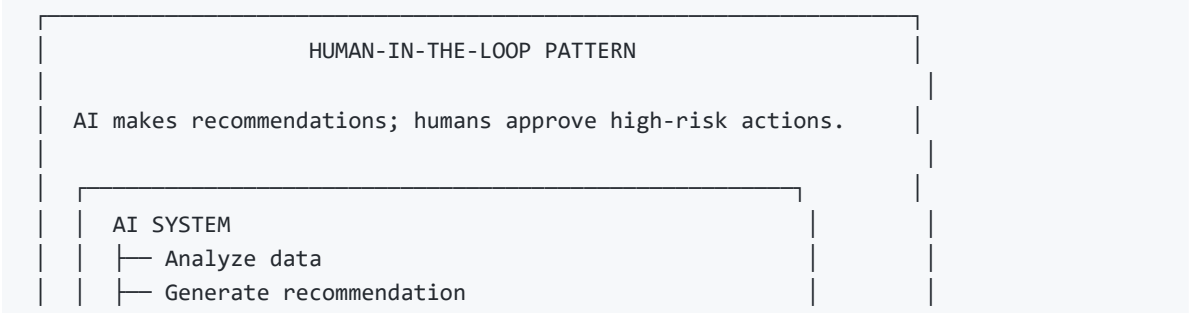
Pattern 2: Predictive Models for Operations (AIOps)

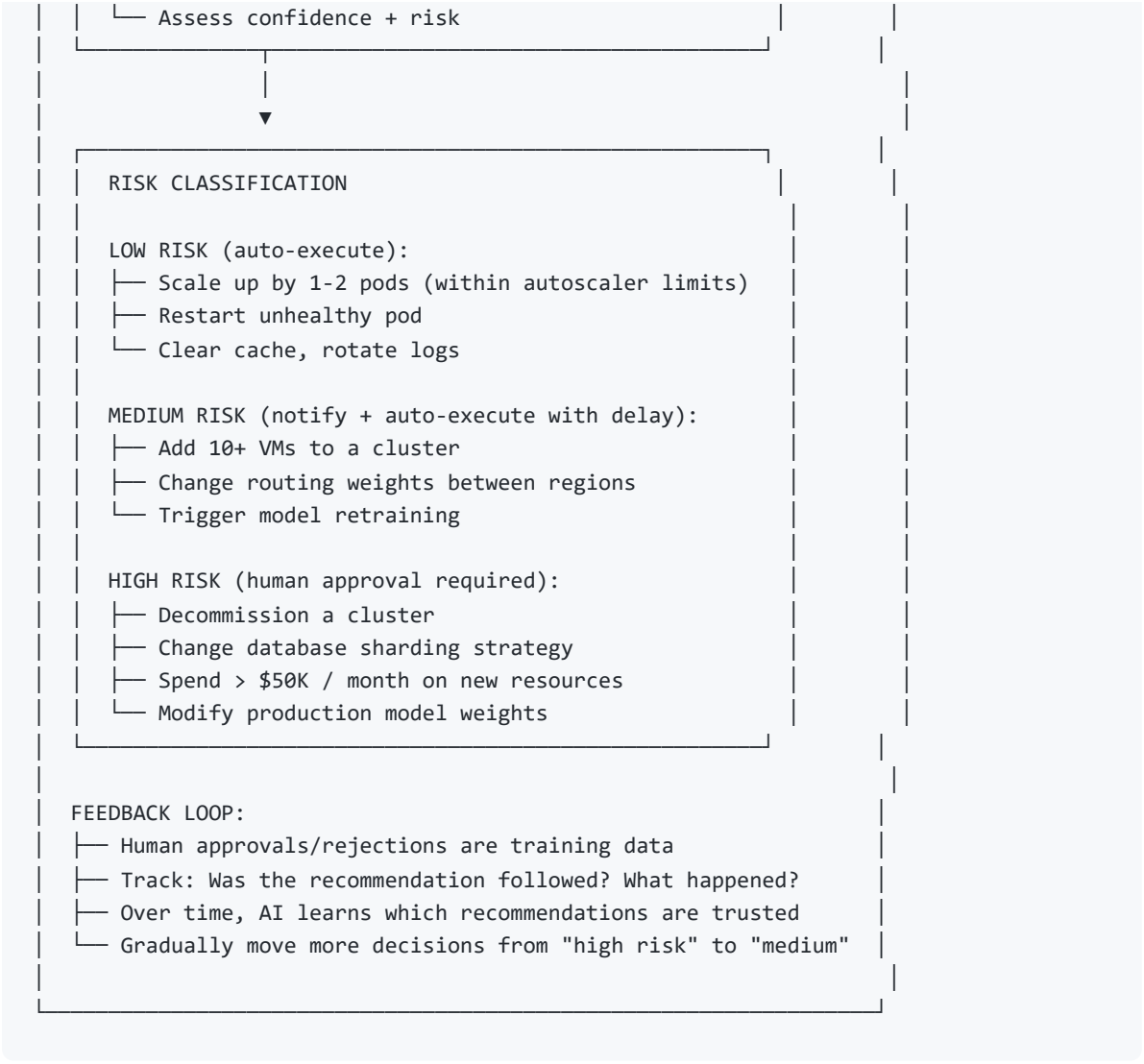
```
┌─────────────────────────────────────────────────────────────────┐
│                   AIOps — PREDICTIVE OPERATIONS                 │
│                                                                 │
│  ANOMALY DETECTION:                                             │
│  ├── Input: Time-series metrics (CPU, latency, error rate)      │
│  ├── Model: Statistical (Z-score, DBSCAN) or ML (Isolation      │
│  │   Forest, autoencoders, Temporal Convolutional Networks)     │
│  ├── Output: "Anomaly detected in API latency at 14:32"         │
│  └── Action: Alert on-call engineer + suggest investigation     │
│                                                                 │
│  INCIDENT PREDICTION:                                           │
│  ├── Input: Metrics + recent changes + calendar (deploy window?)│
│  ├── Model: Gradient boosted classifier trained on past incidents│
│  ├── Output: "75% probability of CPU saturation in 2 hours"     │
│  └── Action: Pre-scale resources or alert team proactively      │
│                                                                 │
│  AUTO-REMEDIATION:                                              │
│  ├── Input: Detected anomaly + known remediation playbooks      │
│  ├── Logic: Match anomaly pattern to playbook → execute         │
│  │   ┌──────────────────────────────────────────────┐          │
│  │   │  IF anomaly_type = "memory_leak"             │          │
│  │   │  AND confidence > 0.9                        │          │
│  │   │  AND blast_radius = "single_pod"             │          │
│  │   │  THEN auto_restart_pod()                     │          │
│  │   │  ELSE escalate_to_human()                    │          │
│  │   └──────────────────────────────────────────────┘          │
│  └── Key: Only auto-remediate LOW RISK actions. Escalate rest.  │
│                                                                 │
│  CAPACITY FORECASTING (ML-enhanced):                            │
│  ├── Traditional: ARIMA on single metric                        │
│  ├── ML-enhanced: Multi-variate model considering:              │
│  │   ├── Historical usage patterns                              │
│  │   ├── Business signals (user growth, marketing campaigns)    │
│  │   ├── Seasonal patterns (holiday shopping, back-to-school)   │
│  │   ├── Known upcoming events (product launches, migrations)   │
│  │   └── Correlated metrics (if storage grows, compute follows) │
│  └── Result: More accurate forecasts with explainable drivers   │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Pattern 3: RAG for Decision Support

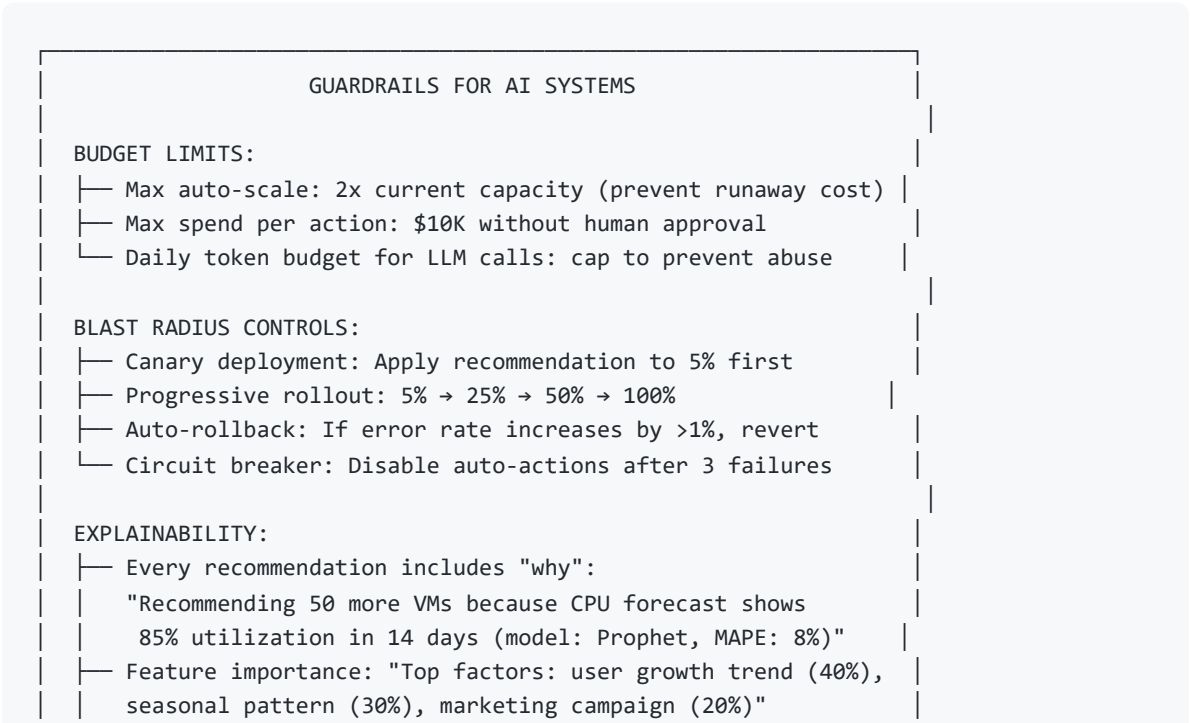


Pattern 4: Human-in-the-Loop (HITL)





Guardrails & Trust in AI-Assisted Decisions



```
│  └── Confidence interval: "We are 95% confident demand will be  │
│      between 70-90% of current capacity"                        │
│                                                                  │
│  AUDITABILITY:                                                   │
│  ├── Log every AI recommendation + human decision               │
│  ├── Track accuracy: What % of recommendations were correct?    │
│  ├── Regular review: Monthly audit of AI-automated actions      │
│  └── Compliance: Meet regulatory requirements for AI decisions  │
└──────────────────────────────────────────────────────────────────┘
```

Interview Talking Points — AI-Assisted Engineering

- "The key principle is **AI augments, not replaces** human judgment. AI handles the data analysis and pattern recognition; humans make the final call on high-risk decisions."
- "I use the **HITL (Human-in-the-Loop) pattern**: classify decisions by risk level. Low-risk actions are auto-executed (pod restart, small scale-up). High-risk actions require human approval (major spending, architecture changes)."
- "**RAG is essential** for grounding LLMs in enterprise context. Without it, LLMs hallucinate about your specific infrastructure. With RAG, the LLM references your actual runbooks, past incidents, and architecture docs."
- "Guardrails are non-negotiable: **budget limits** (cap auto-scaling spend), **blast radius controls** (canary/progressive rollout), **explainability** (every recommendation shows the 'why'), and **auditability** (log every decision for review)."
- "For AIOps, the most impactful pattern is **anomaly detection → incident prediction → proactive scaling**. Instead of reacting to outages, you prevent them. This requires good time-series models and access to leading indicators (business metrics, not just infrastructure metrics)."

Part 4 — Interview Preparation

Terminology Quick Reference

Term	Definition
ARIMA	AutoRegressive Integrated Moving Average — classical time-series forecasting model
Prophet	Facebook/Meta's time-series model — handles seasonality, holidays, missing data

Holt-Winters	Triple exponential smoothing — captures level, trend, and seasonality
MAPE	Mean Absolute Percentage Error — measures forecast accuracy (lower is better)
LP / MIP	Linear Programming / Mixed Integer Programming — optimization techniques
Bin Packing	Optimization problem: fit items into minimum containers
ETL / ELT	Extract-Transform-Load vs Extract-Load-Transform — data pipeline patterns
CDC	Change Data Capture — replicate database changes in real-time
dbt	Data Build Tool — SQL-based transformation framework (industry standard for ELT)
Feature Store	Centralized store for ML features — ensures consistency between training and serving
MLOps	ML Operations — lifecycle management for ML models (train, deploy, monitor, retrain)
Data Drift	Input data distribution changes from what model was trained on
Concept Drift	Relationship between inputs and outputs changes over time
AIOps	AI for IT Operations — anomaly detection, incident prediction, auto-remediation
RAG	Retrieval-Augmented Generation — ground LLM responses in retrieved documents
HITL	Human-in-the-Loop — AI suggests, human approves for high-risk decisions
Shadow Deployment	Run new model alongside old one without serving new results to users
Canary Deployment	Roll out change to small % of traffic first, monitor, then expand
Lakehouse	Architecture combining data lake (cheap storage) and warehouse (SQL, ACID)
PSI	Population Stability Index — measures distribution shift (used for drift detection)

30-Second Pitches

Capacity Planning:

> "Capacity planning is a continuous loop: collect usage metrics, forecast future demand using time-series models like Prophet or ensemble methods, then optimize resource allocation using constraint solvers to minimize cost while meeting SLA targets. The key insight is that **forecasting and optimization are separate concerns** — forecasting tells you how much you'll need, optimization tells you the cheapest way to get it. I always include confidence intervals so decision-makers understand the risk range, not just the point estimate."

Data Platforms & MLOps:

> "A modern data platform flows from **ingestion** (Kafka, CDC) through **storage** (data lake in Delta/Iceberg format) to **processing** (dbt for ELT transformations) to **serving** (warehouse for BI, feature store for ML, APIs for applications). For ML specifically, the lifecycle is: feature engineering → training with experiment tracking (MLflow) → offline validation → deploy via model registry → monitor for data/concept drift → auto-retrain when accuracy degrades. The biggest lesson I've learned is that **feature stores prevent training-serving skew**, which is the #1 silent killer of ML model accuracy in production."

AI-Assisted Engineering:

> "AI-assisted engineering uses LLMs and predictive models to augment decision-making. The three patterns I've found most impactful are: **predictive ops** (anomaly detection + incident prediction for proactive scaling), **RAG for decision support** (LLMs grounded in internal runbooks and past incidents), and **LLM-as-orchestrator** (natural language interface that calls structured APIs for capacity forecasting, what-if simulations, and report generation). The critical design principle is **risk-tiered automation**: auto-execute low-risk actions, require human approval for high-risk ones, and always provide explainability and audit trails."

Common Follow-up Questions

"How would you build a capacity forecasting system from scratch?"

1. **Start simple**: Collect metrics into a time-series DB (Prometheus, InfluxDB). Build dashboards (Grafana).
2. **Add basic forecasting**: Run Prophet/ARIMA on key metrics (CPU, storage, QPS). Generate weekly forecasts with confidence intervals.
3. **Add optimization**: Once forecasts are reliable, build a constraint solver that recommends the cheapest resource allocation meeting SLAs.
4. **Add ML**: Incorporate business signals (user growth, campaigns) as additional features. Use XGBoost or ensemble methods.

5. **Add automation**: Auto-scale based on short-term predictions. Send human-approval requests for large capacity changes.
6. **Add LLM interface**: Natural language queries, automated report generation, what-if simulations.

Key: **Don't start with AI. Start with good data and simple models. Add complexity only when simple models aren't accurate enough.**

"How do you handle model drift in production?"

- **Monitor continuously**: Track prediction accuracy on a sliding window (hourly, daily). Alert when MAPE or accuracy degrades beyond a threshold.
- **Data drift detection**: Compare feature distributions (using PSI or KS test) between training data and recent production data. If distributions diverge significantly, flag for investigation.
- **Concept drift detection**: If accuracy drops but data distributions look stable, the underlying relationships have changed. This is harder to detect and often requires domain knowledge to understand why.
- **Remediation**: Retrain on recent data (window-based or exponential decay). For sudden drift (like COVID), you may need to reset and retrain from scratch.
- **Automation**: Set up a retraining pipeline triggered by drift alerts. Include offline validation before promoting the new model.

"What's the difference between a feature store and a data warehouse?"

- **Data warehouse** (Snowflake, BigQuery): Optimized for **analytical queries** (aggregate, join, filter). Used by analysts and BI tools. Schema designed for business questions.
- **Feature store** (Feast, Tecton): Optimized for **ML feature serving** — both offline (point-in-time correct features for training) and online (low-latency key-value lookups for real-time inference). Schema designed around entity-feature mappings.
- The feature store ensures the **same computation logic** is used in training and serving (no training-serving skew). A data warehouse doesn't solve this problem because training queries and serving lookups have fundamentally different access patterns.

"How do you decide what to automate vs keep human-in-the-loop?"

Use a **risk matrix**:

- **Low risk + high confidence**: Automate fully (pod restarts, small auto-scale, cache clearing)
- **Low risk + low confidence**: Automate with notification (alert the team but proceed)
- **High risk + high confidence**: Automate with delay (execute in 30 min unless human cancels)
- **High risk + low confidence**: Always require human approval (spending decisions, architectural changes, data deletion)

Over time, as you build trust in the system and collect feedback data, gradually move decisions from "human required" to "automated". But always keep the option for humans to override.

"How would you use an LLM to help with capacity planning?"

Three concrete use cases:

1. **Natural language query interface**: "How much GPU capacity will we need in Q4?" → LLM calls the forecasting API with the right parameters, retrieves the forecast, and formats a human-readable answer with charts.
2. **Automated reporting**: LLM pulls weekly metrics, compares against forecasts, identifies notable deviations, and generates a structured capacity report for leadership — saving 2-3 hours of analyst time per week.
3. **What-if simulation interpreter**: "What if we lose the East US region?" → LLM translates this to API calls against the capacity simulator, then explains the impact in business terms ("We'd need to failover 10,000 VMs to West US, increasing latency by 40ms for East Coast users, at an additional cost of $80K/month").

The LLM never does the math or forecasting itself — it **orchestrates** the structured tools that do, and **translates** the results into human-understandable language.

"What's the most common mistake in building data pipelines?"

Not treating data quality as a first-class concern. Teams build pipelines that move data from A to B, but don't validate that:

- All expected records arrived (completeness)
- Values are within expected ranges (accuracy)
- Schema hasn't changed upstream (conformance)
- There are no unexpected duplicates (uniqueness)

When a data quality issue silently propagates, it corrupts ML models, dashboards, and decisions downstream. The fix: **Test data like you test code** — use tools like dbt tests or Great Expectations to define assertions at every pipeline stage. Alert immediately when assertions fail. Never let bad data reach production serving layers.

Trade-off Cheat Sheet

Decision	Option A	Option B	Guidance
Forecasting method	Statistical (ARIMA, Prophet)	ML (XGBoost, LSTM)	**Start statistical**, add ML when you have enough data and correlated features

Pipeline architecture	Lambda (batch + stream)	Kappa (stream only)	**Kappa** if your stream processor can handle backfills; **Lambda** if you need guaranteed batch accuracy
Storage	Data lake	Data warehouse	**Lakehouse** (Delta/Iceberg) — get the best of both
Model serving	Real-time	Batch	**Match to use case** — real-time for < 100ms needs, batch for daily reports
Automation level	Fully automated	Human-in-the-loop	**Risk-based** — automate low-risk, HITL for high-risk
LLM integration	Fine-tuned model	RAG + base model	**RAG first** — cheaper, no training data needed, easier to update. Fine-tune only if RAG quality insufficient
Drift response	Scheduled retraining	Drift-triggered retraining	**Both** — scheduled as baseline, drift-triggered for sudden shifts
Feature computation	Compute at serving time	Pre-compute in feature store	**Feature store** — eliminates training-serving skew, faster serving